Comprehensive Measure of Well-Being Project Workflow

The **Comprehensive Measure of Well-Being Prediction System** is a machine learning-based web application that predicts a country's overall well-being score using important socio-economic and human development indicators. The project follows a structured software development workflow consisting of multiple epics. Each epic represents a major phase of the machine learning lifecycle, from environment setup to model deployment using Flask. This systematic approach ensures that the application is accurate, scalable, maintainable, and easy to understand.

Epic 1: Environment Setup and Package Installation

Objective

Prepare the development environment by installing all the necessary software packages and organizing the project structure.

Story 1: Install Required Python Libraries

Install all libraries required for data analysis, visualization, machine learning, model serialization, and web application development.

The required libraries include:

- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn
- Flask
- Pickle
- Joblib (optional)
- Jupyter Notebook (optional)

These libraries provide functionalities for numerical computation, data manipulation, visualization, machine learning model development, and deployment.

Story 2: Create Project Folder Structure

Organize the project into separate folders for better maintainability.

```
Comprehensive_WellBeing/
│
├── Dataset/
│     well_being.csv
│
├── Models/
│     wellbeing_model.pkl
│
├── Static/
│     style.css
│
├── Templates/
│     index.html
│     result.html
│
├── Images/
│
├── app.py
├── model.py
└── requirements.txt
```

A well-structured directory makes the project easier to manage, maintain, and deploy.

Epic 2: Importing Required Libraries

Objective

Import all necessary Python modules that support every stage of the project.

Story 1: Import Libraries

The project imports libraries including:

- Pandas for dataset handling
- NumPy for numerical operations
- Matplotlib for plotting graphs
- Seaborn for statistical visualization
- LabelEncoder for categorical encoding
- Train-Test Split for dataset division
- Linear Regression for prediction
- Metrics for model evaluation
- Pickle for model saving
- Flask for web application development

Each imported library performs a specific task during preprocessing, model training, evaluation, and deployment.

Epic 3: Dataset Download and Understanding

Objective

Collect the dataset and understand its structure before building the prediction model.

Story 1: Download Dataset

The Comprehensive Measure of Well-Being dataset is downloaded from Kaggle and stored inside the Dataset folder.

The dataset contains socio-economic and development indicators such as:

- Country
- Region
- Life Expectancy
- Mean Years of Schooling
- Gross National Income
- Expected Years of Schooling
- Human Development Index (HDI)
- Happiness Score
- Health Indicators
- Education Indicators
- Economic Indicators
- Comprehensive Measure of Well-Being (Target Variable)

The dataset serves as the foundation for developing the predictive model.

Story 2: Dataset Exploration

The dataset is explored using Pandas functions:

- head()
- tail()
- info()
- describe()
- shape
- columns
- dtypes
- value_counts()

Dataset exploration helps understand:

- Number of rows and columns
- Feature names

- Data types
- Statistical summaries
- Missing values
- Unique categories

This understanding is essential before preprocessing.

Story 3: Data Visualization

Several visualizations are created to analyze relationships between variables.

Common visualizations include:

- Scatter plots
- Strip plots
- Histograms
- Count plots
- Correlation heatmap
- Pair plots
- Box plots

These visualizations help identify:

- Feature distributions
- Correlations
- Outliers
- Trends
- Relationships between predictor variables and well-being score

Visualization provides meaningful insights that improve feature selection.

Epic 4: Data Preprocessing and Label Encoding

Objective

Prepare the dataset for machine learning.

Story 1: Feature Selection

Separate the dataset into:

Independent Variables (X)

Examples:

- Country
- Region
- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- Gross National Income
- HDI

Dependent Variable (Y)

- Comprehensive Measure of Well-Being

Only relevant features are selected for model training.

Story 2: Missing Value Handling

The dataset is checked for missing values using:

```
isnull().sum()
```

If missing values exist, they are handled using:

- Mean
- Median
- Mode
- Removal (if necessary)

Handling missing values improves model accuracy.

Story 3: Label Encoding

Machine learning algorithms cannot directly process text values.

Categorical columns such as:

- Country
- Region

are converted into numeric labels using LabelEncoder.

Example:

```
India      → 0
USA        → 1
Canada     → 2
Germany    → 3
```

Label encoding transforms categorical information into machine-readable numerical values.

Story 4: Final Dataset Preparation

After preprocessing:

- Features are cleaned.
- Missing values are removed.
- Categories are encoded.
- Dataset becomes fully numerical.

The processed dataset is now ready for training.

Epic 5: Splitting the Dataset

Objective

Divide the processed dataset into training and testing datasets.

Story 1: Train-Test Split

The dataset is divided using:

- 80% Training Data
- 20% Testing Data

Training data teaches the model.

Testing data evaluates the model using previously unseen samples.

This prevents overfitting and measures model generalization.

Epic 6: Model Development Using Linear Regression

Objective

Build the prediction model.

Story 1: Train Linear Regression

The Linear Regression algorithm learns relationships between socio-economic indicators and well-being scores.

Training includes:

- Model initialization
- Model fitting
- Learning regression coefficients

The trained model estimates the impact of each feature on overall well-being.

Story 2: Prediction

After training, predictions are generated using:

```
model.predict()
```

The predicted well-being scores are compared with actual values.

Prediction demonstrates how well the model estimates unseen data.

Story 3: Model Evaluation

The prediction model is evaluated using regression metrics such as:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score (Coefficient of Determination)

Additional visualizations include:

- Actual vs Predicted Plot
- Residual Plot

High R^2 values and low error metrics indicate a well-performing model.

Epic 7: Saving the Machine Learning Model

Objective

Store the trained model for future use.

Story 1: Serialize the Model

The trained Linear Regression model is saved using Pickle.

Example:

`wellbeing_model.pkl`

Serialization eliminates the need to retrain the model every time.

Story 2: Store Model

The saved model is placed inside the Models directory.

During deployment, Flask loads this model directly for prediction.

This significantly reduces application response time.

Epic 8: Flask Web Application Development

Objective

Deploy the trained machine learning model through a user-friendly web application.

Story 1: Develop Flask Backend

The Flask application performs the following tasks:

- Accept user inputs
- Load the saved model
- Preprocess input values

- Generate prediction
- Return prediction results

The backend acts as the bridge between the user interface and the machine learning model.

Story 2: Design User Interface

HTML templates provide an interactive interface where users can enter details such as:

- Country
- Region
- Life Expectancy
- Mean Years of Schooling
- Expected Years of Schooling
- Gross National Income
- HDI

After submitting the form, the application displays the predicted **Comprehensive Measure of Well-Being** score.

CSS is used to create a responsive and visually appealing interface.

Story 3: Application Testing and Validation

The complete Flask application is thoroughly tested to ensure:

- Correct input handling
- Accurate predictions
- Proper loading of the trained model
- Error-free execution
- Responsive user interface
- Reliable performance for different user inputs

Testing verifies that the deployed system produces consistent and accurate well-being predictions.
